README

JTAGprobe_smp_pio_std

`JTAGprobe_smp_pio` with a **standard 10-pin layout on the Pico's pins 1–10** (GP0–GP7): UART, SWD/JTAG, resets and two grounds on one row of the header, so one 10-way ribbon reaches everything. The firmware is otherwise the same (same PIO JTAG, same µOS++ SMP split, same USB ids `2e8a:000c` and product string, so the OpenOCD configs work unchanged). See *Pinout Mapping* below.

One more difference: the `DAP_SWJ_*` macros in `include/DAP.h` are again the CMSIS-DAP **protocol bit positions** of the `DAP_SWJ_Pins` command (TCK 0, TMS 1, TDI 2, TDO 3, nTRST 5, nRESET 7), and the probe's GPIO numbers are `PROBE_PIN_*` in `include/DAP_config.h`. In `JTAGprobe_smp_pio` the two shared one set of names holding GPIO numbers (14–21), so `DAP_SWJ_Pins` (OpenOCD's SRST/TRST control) matched no bit and did nothing; with GPIOs 2–7 the same mix-up would have driven the wrong pins.

A CMSIS-DAP v2 JTAG and SWD debug probe firmware with an integrated USB CDC-ACM UART serial bridge for the **Raspberry Pi Pico (RP2040)**, running on the **µOS++ III SMP** kernel across both Cortex-M0+ cores with **RP2040 PIO Hardware Acceleration** for JTAG pins.

It builds on `JTAGprobe_smp` (dual-core µOS++ III SMP kernel) and introduces **hardware state machine acceleration using the RP2040 Programmable I/O (PIO)** blocks: - **Core 0**: Handles USB communication (TinyUSB) and the CDC-ACM UART bridge. - **Core 1**: Dedicated to CMSIS-DAP command execution. - **PIO State Machine**: Generates TCK, shifts TDI, sets TMS and samples TDO for every `DAP_JTAG_Sequence` entry; core 1 only streams the entries into its FIFOs and sorts the TDO bytes that come back.

Features

- **PIO JTAG:** `DAP_JTAG_Sequence`, the command OpenOCD uses in JTAG mode, runs on an RP2040 PIO state machine (`src/jtag.pio` , `src/pio_jtag.c` , `include/pio_jtag.h`), with TCK at the requested `adapter speed` and a 50 % duty cycle while core 1 keeps the PIO's FIFOs fed (when it falls behind, TCK stays low longer). All entries of a command are streamed back to back: the PIO sets TMS from each entry's header, so the CPU never waits between entries.
- **Measured** (Raspberry Pi 4 B, `adapter speed 10000`): the JTAG work per 512-byte packet takes 292 µs instead of 581 µs with CPU bit-banging; `load_image` is 16 % faster and smp_mem_test 5 % faster (see *PIO results* below).
- **JTAG & SWD Debugging:** Full CMSIS-DAP v2 bulk transport support for OpenOCD, pyOCD, probe-rs, and Keil MDK.
- **Hardware USB CDC-ACM UART Bridge:** Virtual COM port (`/dev/ttyACMx` on Linux, `COMx` on Windows) bridged directly to target UART on GPIO 0/1 (Pico pins 1/2).
- **High Performance:**
  - PIO serialization and deserialization of `DAP_JTAG_Sequence` (the other DAP commands still bit-bang on the CPU, see below).
  - Zero blocking debug logging overhead on DAP transfers.
  - Single-cycle RP2040 SIO atomic register fallback for control lines and SWD.
  - Multi-packet ring buffer (4 slots) with 512-byte DAP packet size.
- **Dual-core (SMP):** The DAP thread runs on core 1, overlapping JTAG hardware transfers with USB and UART bridge activity on core 0.
- **Dynamic Clock & Baud Rates:** Automatic PIO clock divider adjustment tracking OpenOCD `adapter speed` commands, and dynamic UART line coding.
- **Supported Hardware:** Raspberry Pi Pico (RP2040).

PIO JTAG

In JTAG mode OpenOCD drives the target almost only with `DAP_JTAG_Sequence` commands: up to a few hundred entries per 512-byte packet, each one 1–64 TCK cycles with a fixed TMS level, TDI bits and an optional TDO capture. `JTAGprobe_smp_pio_std` runs those commands on an RP2040 PIO state machine (`src/jtag.pio` , `src/pio_jtag.c` , `include/pio_jtag.h`).

1. The state machine (`src/jtag.pio`)

```
.program jtag_tdo
.side_set 1                         ; TCK = side-set (GPIO 2), TDI = OUT (GPIO 6),
                                    ; TMS = SET (GPIO 3),      TDO = IN (GPIO 7)
.wrap_target
    pull block          side 1      ; Header, hold TCK high
    out x, 7            side 1      ; X = TCK count - 1
    out y, 1            side 1      ; Y = TMS level
    jmp !y tms_low      side 1
    set pins, 1         side 1      ; TMS high while TCK is high
    jmp bits            side 1
tms_low:
    set pins, 0         side 1      ; TMS low while TCK is high
bits:
    out null, 32        side 1      ; Empty the OSR, so the first `pull ifempty` fetches TDI data
loop:
    pull ifempty block  side 0      ; Next TDI byte after every 8 bits, drive TCK low
    out pins, 1         side 0      ; Shift 1 bit to TDI (2 cycles low)
    in pins, 1          side 1      ; Sample 1 bit from TDO, drive TCK high
    jmp x--, loop       side 1      ; Decrement X and loop (2 cycles high total)
    push block          side 1      ; Push the partial (or empty) last word
.wrap
```

- **Stream protocol.** Per entry the TX FIFO gets a header word (bits 0–6: TCK count − 1, bit 7: TMS) and then one TDI byte per word. The RX FIFO returns `count / 8 + 1` words: the full TDO bytes, then one word with the last `count % 8` bits (empty when the count is a multiple of 8).
- **TMS** is set by the PIO from the header while TCK is still high, before the entry's first falling edge: the target samples it on the rising edges.
- **TDI** is fetched explicitly with `pull ifempty` (autopull off, threshold 8). With autopull, the header would stay in the output shift register and go out as TDI data, and the next entry's header would be prefetched as data.
- **TDO** uses 8-bit autopush with right shift: a full byte arrives in bits 31..24 of the RX word, the last r bits in bits 31..32−r. The final `push block` waits for RX FIFO space instead of dropping a word.
- **Clock.** Every bit takes 4 PIO cycles, 2 with TCK low and 2 with TCK high, so `div = f_sys / (4 × f_TCK)` . `DAP_SWJ_Clock` (OpenOCD `adapter speed`) sets the divider with `pio_jtag_set_clock()` ; `PORT_JTAG_SETUP` restarts the PIO at the last requested speed.

2. The CPU side (`pio_jtag_sequences()`)

`DAP_JTAG_Sequence` (`src/DAP.c`) hands a whole command to `pio_jtag_sequences()` . Two cursors walk the same request: one feeds headers and TDI bytes into the TX FIFO, the other sorts the RX words into the response, keeping only the entries with the TDO flag. The PIO runs entry after entry; core 1 only keeps both 4-word FIFOs moving and never waits for an entry to finish.

3. Pins shared with the CPU bit-banging paths

The other JTAG commands (`DAP_Transfer` , `DAP_TransferBlock` , `DAP_WriteABORT` , `DAP_JTAG_IDCODE`) and `DAP_SWJ_Sequence` / `DAP_SWJ_Pins` still bit-bang through SIO. Their entry points call `PIN_TCK_TDI_TO_SIO()` (`include/DAP_config.h`), which hands TCK, TDI and TMS back to SIO at their current levels, so no pin glitches; the

next `DAP_JTAG_Sequence` returns them to the PIO. SWD mode does not use the PIO at all (`PORT_SWD_SETUP` stops it). TDO has the input synchronizer bypassed (`input_sync_bypass`); nTRST and nRESET stay on SIO.

4. PIO results (Raspberry Pi 4 B, smp_mem_test, `SEMI_PRINTS=1`, 512-byte packets)

Probe firmware	`adapter speed`	`load_image` (74,888 B)	JTAG work per packet	Load cycle per packet	Whole test (CTest)
JTAGprobe_smp, CPU bit-banging	4000	1,528 ms (49.0 KB/s)	581 µs	1,745 µs	308.1 s
PIO, TMS through SIO, waiting after each entry	4000	1,702 ms	755 µs	1,934 µs	324.5 / 332.0 s
PIO, TMS through SIO, waiting after each entry	10000	1,578 ms	621 µs	1,794 µs	315.3 s
PIO, TMS in the stream (current)	**10000**	**1,286 ms (58.2 KB/s)**	**292 µs**	**1,450 µs**	**291.5 s**
PIO, TMS in the stream (current)	15000	1,287 ms (58.2 KB/s)	291 µs	1,448 µs	285.1 s

All runs **PASS**; each configuration ran once (earlier identical PIO runs differed by ~8 s in total time). *JTAG work per packet* is the probe's `DAP_ExecuteCommand()` time, from its packet counters (DAP vendor command `0x80`).

- **A PIO that waits after every entry is slower than CPU bit-banging.** With a few hundred short entries per packet, setting TMS from the CPU and waiting for each entry cost ~530 µs per packet; raising TCK from 4 to 10 MHz saved only 134 µs.
- **Streaming the entries, with TMS in the stream, halves the JTAG work** (581 → 292 µs per packet at 10 MHz): `load_image` is 16 % faster than with CPU bit-banging and 35 % faster than with 64-byte packets (1,954–1,994 ms), smp_mem_test 16.6 s faster.
- **15 MHz loads exactly as fast as 10 MHz** (1,287 ms, 291 µs per packet): the TCK clock is no longer the limit; most likely core 1, which feeds the FIFOs one byte per word and sorts the replies, is (not measured separately). The whole test took 285.1 s against 291.5 s, within the ~8 s spread seen between identical runs.
- **What is left** in a load cycle is mostly receiving the 512-byte request (one 64-byte USB packet at a time) and the PC's turnaround: ≈ 920 µs of 1,450 µs. With `SEMI_PRINTS=1` the whole test is still dominated by OpenOCD reading each semihosted string byte by byte (~0.9 s per line).
- Not measured: the current build at 4 MHz.

Both smp_mem tests, built from the GitHub workspace of µOS++ III (`SEMI_PRINTS=1` , PIO with TMS in the stream, `adapter speed 15000`), against the 512-byte CPU bit-banging build at 4 MHz (§ 9.6 of `PERFORMANCE_ANALYSIS_AND_OPTIMIZATION.md`):

Test	Loaded bytes	`load_image`	JTAG work per packet	Whole test (CTest)	CPU bit-banging, 4 MHz
smp_mem_test	75,208	**1,273 ms (59.1 KB/s)**	291 µs	**282.9 s, PASS**	1,528 ms, 308.1 s
smp_mem_inter_test	81,048	**1,399 ms (57.9 KB/s)**	291 µs	**537.4 s, PASS**	1,644 ms, 561.8 s

These images are a few hundred bytes larger than the earlier ones (75,208 against 74,888 bytes): the `SEMI_PRINTS=1` code. Both tests run 4–8 % faster than with CPU bit-banging; smp_mem_test repeated at 15 MHz in 285.1 s (golden build) and 282.9 s (GitHub build). OpenOCD logged `error handling USB events: System call interrupted` and `CMD_DAP_JTAG_SEQ failed` once, right after smp_mem_inter_test's `RESULT: PASS`, most likely the runner stopping OpenOCD in the middle of a command (not investigated further).

µOS++ III SMP Integration

Source layout

Path	What it is
`micro-os-plus/micro-os-plus-iii/`	The µOS++ III SMP kernel, unmodified (a `git archive` of `micro-os-plus-iii`, branch `smp`, commit `931f1380`). CMake INTERFACE library `micro-os-plus::iii`.
`micro-os-plus/micro-os-plus-iii-rp2040/`	The RP2040 port, an INTERFACE library `micro-os-plus::rp2040` on top of the kernel, built the way the `micro-os-plus-iii-smp` tests link `micro-os-plus::cortexm`.
`src/cmsis-plus/os-app-config.h`	The kernel configuration: 1000 Hz tick, stack sizes.
`src/probe_os.h`	The objects the threads share: `usb_mutex`, `usb_sem`, `dap_sem`, `uart_sem`.
`CMakeLists.txt`	Pico SDK project; `add_subdirectory(micro-os-plus/micro-os-plus-iii-rp2040)` and links `micro-os-plus::rp2040`.

The RP2040 port

Item	Implementation
CPUs	`OS_NCPU=2`, `OS_USE_SMP_SCHEDULER=1`
Context switch	ARMv6-M PendSV, lowest priority, one per core
Tick	SysTick on both cores, 1 kHz; only core 0 advances the system clock
Kernel lock	SIO hardware spinlock `PICO_SPINLOCK_ID_OS1` (14), the one the Pico SDK reserves for an RTOS
Critical sections	PRIMASK
Cross-core IPI	SIO inter-core FIFO, `SIO_IRQ_PROC0` / `SIO_IRQ_PROC1`
High-resolution clock	`time_us_64()` (1 MHz timer)
Core 1 start	`os_rp2040_start_core1()` → `multicore_launch_core1_raw()` on the SDK's core-1 stack, then an `idle1` thread
Free store	`__end__` … `__HeapLimit` (the SDK linker script's heap)
Process exit	The Pico SDK's `exit()`; the port removes the kernel's `src/libc/stdlib/exit.c` from the build

Threads

Thread	Priority	Core	Job
`UART`	high	0	CDC ↔ `uart0` bridge (must preempt USB, or characters are lost)
`TUD`	above normal	0	`tud_task()`, then services the DAP endpoints
`DAP`	normal	1	`DAP_ExecuteCommand()`: the JTAG/SWD bit-banging
`main`	—	0	`os_main()`: probe init, creates the threads, starts core 1

tinyusb is built with `OPT_OS_NONE`; every tinyusb call is made with `usb_mutex` held. The DAP requests and responses cross between the cores in two lock-free single-producer/single-consumer rings (`src/tusb_edpt_handler.c`). The DAP thread busy-waits up to 2 ms for the next packet before it sleeps, and starts the IN transfer of each response itself when it gets `usb_mutex` (otherwise the `TUD` thread does it right after `tud_task()`). The `TUD` thread likewise keeps polling `tud_task()` for 2 ms after the last DAP packet instead of sleeping.

DAP packets are **512 bytes** (`DAP_PACKET_SIZE`, CMSIS-DAP v2) over the 64-byte full-speed bulk endpoint, so OpenOCD packs eight times more into each round trip. A request arrives as up to eight USB packets; since the host sends no ZLP, `dap_req_len()` parses the DAP command lengths to find where a request ends on a full 64-byte packet, and a response of a 64-byte multiple shorter than 512 bytes is followed by a ZLP.

How µOS++ III SMP is used

- **One kernel, two cores.** `main()` starts on core 0 under the kernel; `os_rp2040_start_core1()` brings core 1 into the same SMP scheduler, with its own idle thread.
- **Threads pinned to cores.** `create_thread()` in `src/main.c` sets each thread's priority and CPU affinity (`th_cpu_affinity`): `UART` and `TUD` on core 0, `DAP` alone on core 1. On core 0 the scheduler preempts by priority, so the UART bridge always runs ahead of USB; on core 1 nothing ever interrupts the JTAG work.
- **Kernel objects for synchronization.** `usb_mutex` makes every tinyusb call exclusive (tinyusb is not thread-safe, and both cores call it). The binary semaphores `usb_sem`, `dap_sem` and `uart_sem` wake a sleeping thread when it has work: a USB event wakes `TUD`, a new request wakes `DAP`, UART data wakes `UART`.
- **The packet path stays outside the kernel.** During a transfer, requests and responses pass through the two lock-free rings and the threads spin instead of sleeping. Sleeping and waking across cores for every packet is what made the first SMP build 4× slower. The kernel only puts a thread to sleep, and wakes it, when the probe goes idle and becomes active again.

How the two cores are used

Core	Threads	Work during a load (512-byte packets, probe counters)
0	`UART`, `TUD`	USB events while a request comes in and a response goes out, ~7 µs to start each response, and the UART bridge whenever the console has data
1	`DAP`	~580 µs of JTAG bit-banging in each ~1,740 µs packet cycle (about ⅓); the rest it spins, waiting for the next request

For the other ~⅔ of each cycle neither core has useful work: they wait on the USB transfers and the PC's turnaround. So the Pico's CPUs are not the bottleneck; the round trip to the PC is. After 2 ms without packets both threads sleep on their semaphores, and an idle probe uses almost no CPU. (These shares come from the per-stage packet timing; no per-core CPU-load counter was measured.)

Packet timing

The firmware keeps per-stage packet timing (OUT → DAP start, DAP execution, response → IN, IN transfer, OUT → OUT, OUT stalls), read from OpenOCD with the DAP vendor command `0x80`:

```
cmsis-dap cmd 0x80 0xff     ;# reset the counters
cmsis-dap cmd 0x80 <i>      ;# read counter i (0..11), printed as "Returned data b0 b1 b2 b3"
```

The counters and a measured breakdown are in `PERFORMANCE_ANALYSIS_AND_OPTIMIZATION.md` § 8.5 and § 9.4.

OpenOCD Load Times (Raspberry Pi 4 B over JTAG)

`load_image` of µOS++ test ELFs into a Raspberry Pi 4 B (BCM2711) over JTAG, OpenOCD `adapter speed 4000`, CMSIS-DAP v2 bulk, timed around `load_image` alone. See `PERFORMANCE_ANALYSIS_AND_OPTIMIZATION.md` § 9 for every measurement and the per-test table.

Probe firmware	Measured rate	~80 KB image
Stock debugprobe 2.0.0	~1.05 KB/s	~77 s
JTAGprobe `jtagprobe-new` (FreeRTOS, single core)	34.0–39.5 KB/s	2.0–2.4 s
JTAGprobe_smp, first build (DAP thread slept per packet; fixed in the second)	9.95 KB/s	8.1 s
JTAGprobe_smp, second build (DAP thread spins, sends its own responses)	37.8–38.1 KB/s	2.1 s
JTAGprobe_smp, third build (+ `TUD` spin, packet timing)	37.6–38.5 KB/s	2.1 s
JTAGprobe_smp, current build (+ 512-byte DAP packets)	**48.4–49.3 KB/s**	**1.65 s**

With 64-byte packets the SMP build loads at the speed of the optimized single-core firmware (about 3.8× faster than the first SMP build). Its packet counters showed the core-to-core hand-off costs ~7 µs of a ~263 µs cycle; the rest was the JTAG bit-banging (~68 µs) and the USB round trip with the PC (~188 µs). So the current build moves 512-byte packets: 8.6× fewer round trips, loads **23 % faster** (about 47× the stock firmware), and all tests pass faster:

hwd test, Pi 4 B	64-byte packets	512-byte packets
smp_mem_test	50.2 s	**47.1 s**
smp_mem_inter_test	186–193 s	**175.9 s**
smp_mem_test, `SEMI_PRINTS=1`	337.4 s	**308.1 s**
smp_mem_inter_test, `SEMI_PRINTS=1`	620.5 s	**561.8 s**

The load gains less than the 8× fewer packets: the bit-banging per byte is unchanged, and receiving a request in 64-byte USB packets plus the PC's turnaround now takes ≈ 910 µs of a ≈ 1,740 µs cycle (not yet split, see `PERFORMANCE_ANALYSIS_AND_OPTIMIZATION.md` § 9.6). With semihosting prints on (`SEMI_PRINTS=1`) each semihosted line still costs ~0.9 s, OpenOCD reading each string byte by byte.

Pinout Mapping

Connect the probe to your target board as follows:

Pico Header Pin	RP2040 GPIO	Signal	Hardware Role / Direction	Description
Pin 1	**GPIO 0**	**UART_TXD**	Output (Probe → Target, `uart0`)	to the target's UART RX
Pin 2	**GPIO 1**	**UART_RXD**	Input (Probe ← Target, `uart0`)	from the target's UART TX
Pin 3	GND	**GND**	Ground Reference	Common Ground
Pin 4	**GPIO 2**	**SWCLK / TCK**	Output (Probe → Target, PIO Side-Set 0)	SWD Clock / JTAG Test Clock
Pin 5	**GPIO 3**	**SWDIO / TMS**	Bi-directional (SWD: SIO GPIO; JTAG: PIO SET 0)	SWD Data I/O / JTAG Test Mode Select
Pin 6	**GPIO 4**	**nTRST**	Output (Probe → Target, SIO GPIO)	JTAG Test Reset (optional)
Pin 7	**GPIO 5**	**nRESET**	Open-drain (Target Reset, SIO GPIO)	System Reset
Pin 8	GND	**GND**	Ground Reference	Common Ground
Pin 9	**GPIO 6**	**TDI**	Output (Probe → Target, PIO OUT 0)	JTAG Test Data In
Pin 10	**GPIO 7**	**TDO**	Input (Probe ← Target, PIO IN 0)	JTAG Test Data Out

USB Interfaces & Serial Port

When plugged into a host PC, `JTAGprobe_smp_pio_std` enumerates as a composite USB device:

1. **Interface 0 (Vendor Specific Bulk):**
   - CMSIS-DAP v2 interface used by OpenOCD, pyOCD, and debuggers via `libusb`.
   - *Note:* CMSIS-DAP is **not** a serial/tty device and will not appear under `/dev/tty*`.
2. **Interfaces 1 & 2 (CDC-ACM UART Bridge):**
   - Creates a **`/dev/ttyACMx`** on Linux (`/dev/ttyACM0` if it is the only one; or `COMx` on Windows).
   - In system logs (`dmesg`, `/dev/serial/by-id/`), the port inherits the USB product string `JTAG probe on Pico (CMSIS-DAP)`. **It is the serial console port connected to GPIO 0 & 1 (Pico pins 1 & 2).**

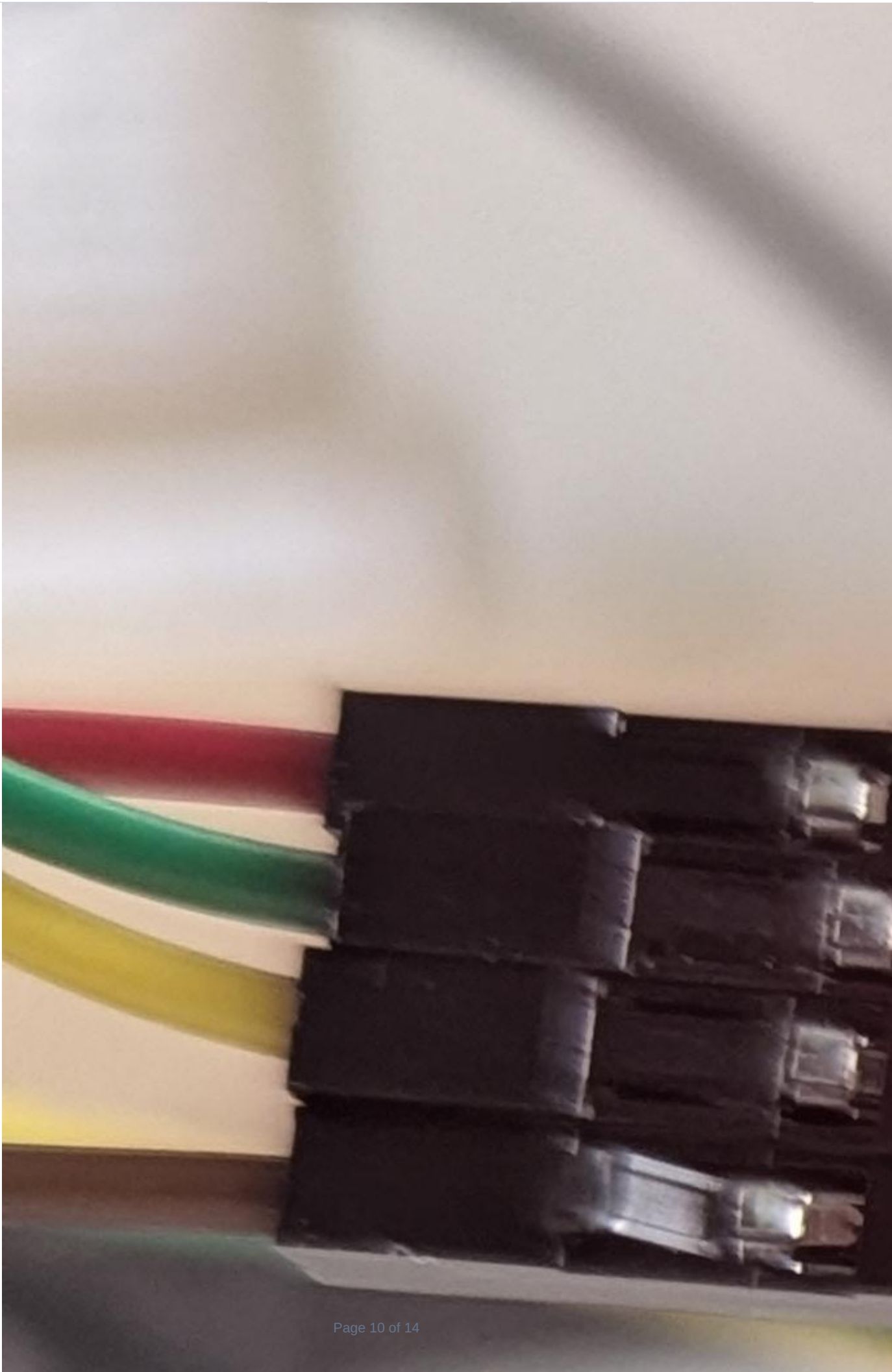
Opening the UART Console

Use any serial terminal application (e.g. `picocom`, `minicom`, `screen`, PuTTY):

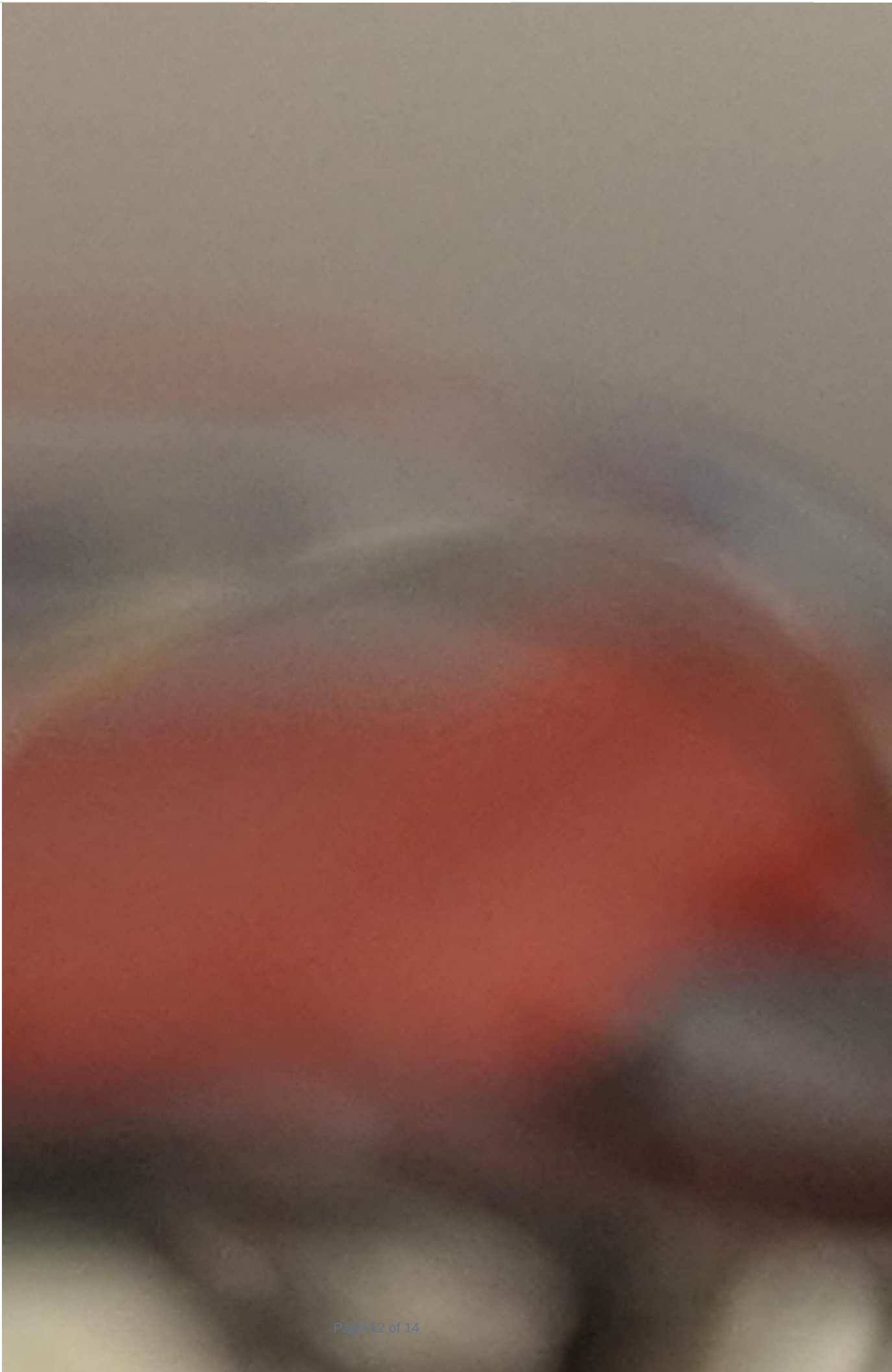
```
# Using picocom at 115200 baud
picocom -b 115200 /dev/ttyACM0

# Using minicom
minicom -b 115200 -o -D /dev/ttyACM0
```

Images of debugging PICO with another PICO and Bluepill



Bluepill Debug



Pico Debug

Building from Source

1. Prerequisites

- **Raspberry Pi Pico SDK** — Set the `PICO_SDK_PATH` environment variable:

```
export PICO_SDK_PATH=/path/to/pico-sdk
```

- **CMake** (3.20 or higher; the µOS++ CMake files need it)
- **GCC ARM Embedded Toolchain** (`arm-none-eabi-gcc`)
- **Ninja** (or `make`)
- **picotool** (found by the SDK, e.g. `-Dpicotool_DIR=$HOME/.local/lib/cmake/picotool`)

The µOS++ kernel and its RP2040 port are already in `micro-os-plus/` ; nothing else needs fetching.

2. Building the Firmware (Raspberry Pi Pico, RP2040)

From the project root directory:

```
cmake -B build -G Ninja -DPICO_BOARD=pico
ninja -C build
```

The output binaries generated in `build/` are: - **`build/JTAGprobe_smp_pio_std.uf2`** — UF2 firmware file for flashing via USB bootloader. - **`build/JTAGprobe_smp_pio_std.elf`** — ELF binary with debug symbols.

The firmware is linked `copy_to_ram` : it runs entirely from SRAM, bypassing XIP flash latency.

Flashing the Firmware

1. **Hold the BOOTSEL button** on your Pico board.
2. **Plug the board into your computer's USB port** while holding BOOTSEL (the device mounts as a mass-storage drive named `RPI-RP2`).
3. **Copy** **`build/JTAGprobe_smp_pio_std.uf2`** to the drive.
4. The Pico will automatically reboot and start running `JTAGprobe_smp_pio_std` with CMSIS-DAP v2, PIO hardware acceleration, and USB CDC UART on GPIO 0/1.

Final Reflection: One CPU vs Two CPUs

Measured on a Raspberry Pi 4 B target over JTAG with OpenOCD 0.12, comparing the optimized single-core FreeRTOS JTAGprobe (`jtagprobe-new`) with JTAGprobe_smp (µOS++ III SMP, DAP on core 1, USB and UART on core 0), both at `adapter speed 4000` . The PIO results are in *PIO JTAG* above and in `PERFORMANCE_ANALYSIS_AND_OPTIMIZATION.md` § 9.7. Details in `PERFORMANCE_ANALYSIS_AND_OPTIMIZATION.md` § 9 and § 11.

	1 CPU (FreeRTOS)	2 CPUs (JTAGprobe_smp)
`load_image`, smp_mem_inter_test (80,984 B)	2,380 ms (34.0 KB/s)	2,106–2,143 ms (37.8–38.5 KB/s)
Probe time per packet (≈ 9.9 loaded bytes)	251–292 µs	259–264 µs
Whole smp_mem_test (CTest, reset wait included)	140 s	50–54 s
Whole smp_mem_inter_test	190 s	186–193 s

Both columns use 64-byte DAP packets and CPU bit-banging. The 512-byte packets (JTAGprobe_smp: 1,644 ms, 47.1 s and 175.9 s for the same three rows) and the PIO (smp_mem_test load 1,286 ms at 10 MHz) cut the time per byte in ways that would help a single-core firmware just as well, so they are left out of this one-vs-two comparison.

JTAG operations (bulk, e.g. `load_image`). Two CPUs give about the same speed as one: the load is a strict request/reply exchange (OpenOCD sends one packet, ~9.9 loaded bytes, and waits for its answer). Of a ~263 µs cycle the probe's counters show ~68 µs of bit-banging, ~7 µs of hand-off between the two cores and ~188 µs of USB transfer and PC turnaround. A second core cannot overlap anything in a chain where each step waits for the previous one; it can only cost time if the hand-off sleeps. The first SMP build did exactly that (two cross-core wake-ups per packet, ~1 ms per packet, 4× slower; history, fixed since the second build), and keeping the packet path free of sleeps (the DAP thread spins and sends its own reply) brought it back to single-core speed.

JTAG operations (scattered, e.g. a running test). Here two CPUs pay off: smp_mem_test, whose run is OpenOCD polling the four Cortex-A72 cores between short exchanges, finished in 50–54 s instead of 140 s. The DAP work on core 1 never waits behind USB event handling or the UART on core 0. (The exact share of each factor was not profiled.) Where the test itself dominates (smp_mem_inter_test, ~170 s of computing on the Pi), one or two probe CPUs make no measurable difference: 186–193 s against 190 s, within its run-to-run spread.

UART. On one CPU the UART bridge task and the DAP task share a core, so the bridge has to preempt the bit-banging to avoid losing characters. On two CPUs the UART thread runs on core 0 next to USB and never delays a DAP command, and no DAP burst can delay the UART. The tests ran with the full console on the UART and passed; the bridge throughput itself was not benchmarked.

Semihosting. To the probe a semihosting call is just more JTAG traffic, so it benefits like any scattered JTAG work. But its cost is set by OpenOCD: `SYS_WRITE0` reads the string from the target one byte per JTAG access, about 0.9 s per console line. With every core-0 line sent through semihosting (`SEMI_PRINTS=1`) smp_mem_test takes 337 s and smp_mem_inter_test 621 s, against 50–54 s and 186–193 s without. Neither one nor two probe CPUs change that; reading the string in blocks on the host would. On this board OpenOCD also serves semihosting only from core 0 of the target: a call from cores 1–3 leaves that core halted.

Where the next gains are. 1. **Fewer round trips**: a larger `DAP_PACKET_SIZE` (**done**, 512 bytes, loads 23 % faster and tests 6–10 % faster); 2. **Faster bit-banging (PIO)**: **done in `JTAGprobe_smp_pio`**: with TMS in the PIO stream and `adapter speed 10000`, the JTAG work per packet halves (581 → 292 µs) and smp_mem_test with `SEMI_PRINTS=1` runs in 291.5 s instead of 308.1 s; 3. **The request reception and the PC's turnaround**: now ≈ 920 µs of each ≈ 1,450 µs load cycle, the largest part; to be measured separately (the OUT endpoint is armed one 64-byte USB packet at a time); 4. **Host-side optimizations**: reading semihosting strings in blocks instead of byte by byte.